

Lab10 进一步的loop优化

1. 实验目标

把循环里“乘加型”的 derived induction variable 转成“加减型”递推，减少每次迭代的高代价运算。
核心等价变换是把： $j = a \cdot i + b, i_{k+1} = i_k + \Delta$ 改写为 $j_{k+1} = j_k + a \cdot \Delta$ 。并在循环头用新 phi 维护 j 的递推值。

2. main.cc

1. 读取带 flow 信息的 SSA 输入（.4-ssa-withflow-xml.quad）。
2. 先做分析与报告（打印 basic IV、derived IV、strength reduction plan）。
3. 再做两阶段优化：
 1. strength reduction pass
 2. cleanup pass（删掉替换后无用的归纳变量链）
4. 输出优化后的 .4-ssa-loopivopt.quad。

3. 数据结构

3.1 BasicInductionVar

定义在 loopinductionbasic.hh。记录了 phi temp、初值、回边值、step（常量或 loop-invariant temp）、更新顺序等。

3.2 DerivedInductionVar

定义在 loopinductionderived.hh。把 derived 统一表示成 affine 形式：basicCoeff, constant，对应为表达式中的系数 a, b 。

3.3 StrengthReductionPlan

定义在 loopstrengthreduction.hh。先生成计划（新 temp、新 phi、初始化表达式、回边更新表达式、插入位置），再统一应用到 IR。

4. loopinductionopt.cc

4.1 loopInductionStrengthReductionPass

克隆程序 -> 对每个函数发现循环头 -> 识别 basic/derived induction variable -> 生成 strength reduction plan -> 应用改写。

4.2 loopInductionCleanupPass

克隆程序后，对每个函数多轮调用无用归纳变量删除，清掉改写后残留的死链条和冗余语句。

5. loopinductionbasic.cc

5.1 bool parseBasicUpdate

功能：识别 basic IV 的更新语句形状，并提取步长。支持的模式：

1. $i = i + c$
2. $i = c + i$
3. $i = i - c$
4. $i = i + t_inv$
5. $i = t_inv + i$
6. $i = i - t_inv$

5.2 discoverBasicInductionVars

功能：主入口，按 loop header 发现 basic induction variables。

遍历每个 loop header，检查 PHI 参数，对于其中位于 loop body 内的 temp，验证他的 def 语句是否为 `MOVE_BINOP`。是则再调用 `parseBasicUpdate` 判断是否为合法的 basic 更新。合法则构造 `BasicInductionVar`，并放入 `map<headerLabel, vector<BasicInductionVar>>result` 中。最终得到每个 Label 对应的块中有哪些 `BasicInductionVar`。

5.3 classifyRelatedTemps

功能：把 basic IV 相关 temp 标成 useful 或 useless。

若 step 是 temp，先把该 temp 加入 related 集合。

遍历 related temps 做使用分析。phi temp 和 init temp(phi 参数中来自 loop body 外的那个) 直接标 useful。未知的标 useful。检查 useSet：只要出现 basicIV 中 phi/update 之外的使用点，就标 useful；
否则标 useless。

6. loopinductionderived.cc

6.1 AffineCandidate

功能：内部中间表示，表示某个 temp 当前可写成 $a \times basicIV + b$ 。

字段意义：

1. basicTempNum 对应基准 basic IV。
2. coeff 和 constant 对应 a 和 b。
3. currentTempNum 是当前候选 temp。
4. defOrder 和 sourceOrder 用于顺序判定。
5. defStm 是定义该候选的语句。

6.2 buildAffineFromBinop

功能：识别并合成 Affine。

支持的主要模式：

1. affine + const
2. const + affine
3. affine - const
4. const - affine
5. affine * const
6. const * affine

不支持两个 affine 操作，因为可能底层的 basicIV 不同，处理起来会比较复杂。

6.3 discoverDerivedInductionVars

逐 loop header 处理，寻找其中的 basic IV，初始化为 affine。然后遍历 loop body blocks，取其中的 definedTemp，如果定义的是 basic IV，则跳过避免将其覆盖为 derived IV。然后针对 `MOVE/MOVE_BINOP` 分别处理，调用 `buildAffineFromBinop`。

最后将 `map<int, AffineCandidate>affineByTemp` 中的 Affine 进行筛选过滤，去除掉属于 basic IV 的，以及 temp 没有被使用过的；并且至少要有一处 use 是在 Affine 链之外的 Affine temp 才保留（否则只是作为一个传递的中间量，可以直接优化掉）。最终封装为 `DerivedInductionVar` 放入 result 中。

7. loopstrengthreduction.cc

先生成替换计划（generate），再把计划落到 IR（apply）。

7.1 generateStrengthReductionPlan

不直接改 IR，先形成一份可执行改写计划 plan。

按 header 遍历 derived，找到对应 basic，填充 ReplacementIV（新 temp、init 表达式、step 表达式、插入标签、删哪些旧语句、替换映射）。`sourceAfterBasicUpdate` 判断 derived 用的是更新前 i 还是更新后 i，从而把 init 和条件常量改写到严格等价。

分配新 temp: `newPhiTemp`, `newInitTemp`, `newBackedgeTemp`。

再根据是否需要中间值补分配：`sourceAfterBasicUpdate` 时分配 `newInitAdjustedSourceTemp`；当 `coeff` 不等于 1 且 `constant` 不为 0，分配乘法中间 temp。

根据 basic 步长类型构建 step 计划：

步长是 temp：

1. 记录 `basicStepTempNum`。
2. 计算缩放后符号与倍率 `signedScale`。
3. 若倍率为 1，直接用原 step temp。
4. 否则分配 `newStepTemp`，表示 `a乘stepTemp`。

步长是常量：直接计算 `stepIncrementValue` 等于 `coeff` 乘 `basic.step`。

寻找插入位置：init 语句调用 `findPreheaderLabel`，update 语句调用 `findBackedgeLabel`。

最后将下列内容写入 plan 并返回：

1. `tempReplacement`：旧 derived temp -> 新 phi temp。
2. `stmtsToRemove`：标记原 derived 定义语句删除。
3. `replacements`：保存完整 repl 供 apply 阶段插入语句。

7.2 applyStrengthReduction

把 generate 阶段的 plan 真正改写到函数 IR 上。

首先遍历所有函数，调用 `replaceTempsInUses` 把 stmt 中的 use 集合中包含 derived temp 的替换为新的 temp。然后遍历每个 replacement，调用 `buildInitStatements(repl)` 得到一串初值语句，放到 `initStmtsByLabel` 对应桶；调用 `makePhi` 创建新的 Phi 语句放到 `phiStmtsByHeader`；调用 `buildUpdateStatement(repl)`，放到 `updateStmtsByLabel` 桶中暂存。最后调用 `rewriteLoopCondition` 把循环条件从比较 basic 改为比较 newPhi。

然后再对桶中暂存的语句统一重写。首先删除 derived 变量的定义，然后在对应块的对应位置插入新 phi, init 和 update。这样做先按照 replacement 顺序遍历，得到全部需要修改的语句，然后再按块顺序进行遍历修改。对于一个 replacement 涉及多个块的修改的情况，确保删除/插入后得到的语句顺序稳定。

8. loopinductionopt.cc

进行清理，删除无用内容。

8.1 isPureDefStm

判断一条语句是否可安全删除。

当前判定为 true 的是 MOVE、LOAD、MOVE_BINOP、PHI、PTR_CALC。

判定为 false 的是有副作用或控制流语义的语句，比如 STORE、CALL、EXTCALL、CJUMP、RETURN 等。

8.2 eliminateUnusedInductionVars

先扫描全函数，建立三类集合：

1. pureDefs：每个纯定义 temp 对应哪条定义语句。
2. pureDeps：每个纯定义 temp 依赖了哪些 used temps。
3. liveTemps：来自非纯定义语句的 used temps，它们用到的值必须保留。

然后进行迭代：初始把 liveTemps 入队。然后每次取队首 temp，将它依赖的所有 temps 标注为 liveTemp 并入队。直到队列为空。

最后遍历删除无用语句：如果是纯定义语句，且定义的 temp 不在 liveTemps，就删除；否则保留。最后回写 block 的语句列表。

9. git graph

传出的更改 dev

hw10 finished Starw1sh

dev

Merge branch 'master' into dev Starw1sh

Merge remote-tracking branch 'origin/...

HW11 added 王晓阳

hw9 handin Starw1sh

mycode/dev

hw9 finished Starw1sh

Merge branch 'master' into dev Starw...

Merge remote-tracking branch 'origin...

HW10: test fmj updated xywang2772

HW10/vendor/genAndrunQuad/READ...

HW10: simplified. README changed xy...

HW10: simplified xywang2772

HW10: simpler test caseswith reports ad...

HW10: simplified xywang2772

HW10: simplified xywang2772

HW10: add genAndrunQuad files for lin...

HW10: add genAndrunQuad 王晓阳

HW10: added helper programs to gener...

HW10 added 王晓阳

Merge branch 'master' into dev Starw...

Merge remote-tracking branch 'origin...

HW9 added 王晓阳